Move Element To End

AlgoExpert · Medium · Arrays

The problem

- In place, move every occurrence of `toMove` to the end (other order irrelevant).

Intuition

- Two pointers: move the target from the left end out of the way by swapping with non-targets from the right.

Brute force vs optimal

- Build a new list: non-targets then targets → O(n) space
- Bubble targets one step at a time → $O(n^2)$
- Two pointers in place → O(n), O(1) space

Approach

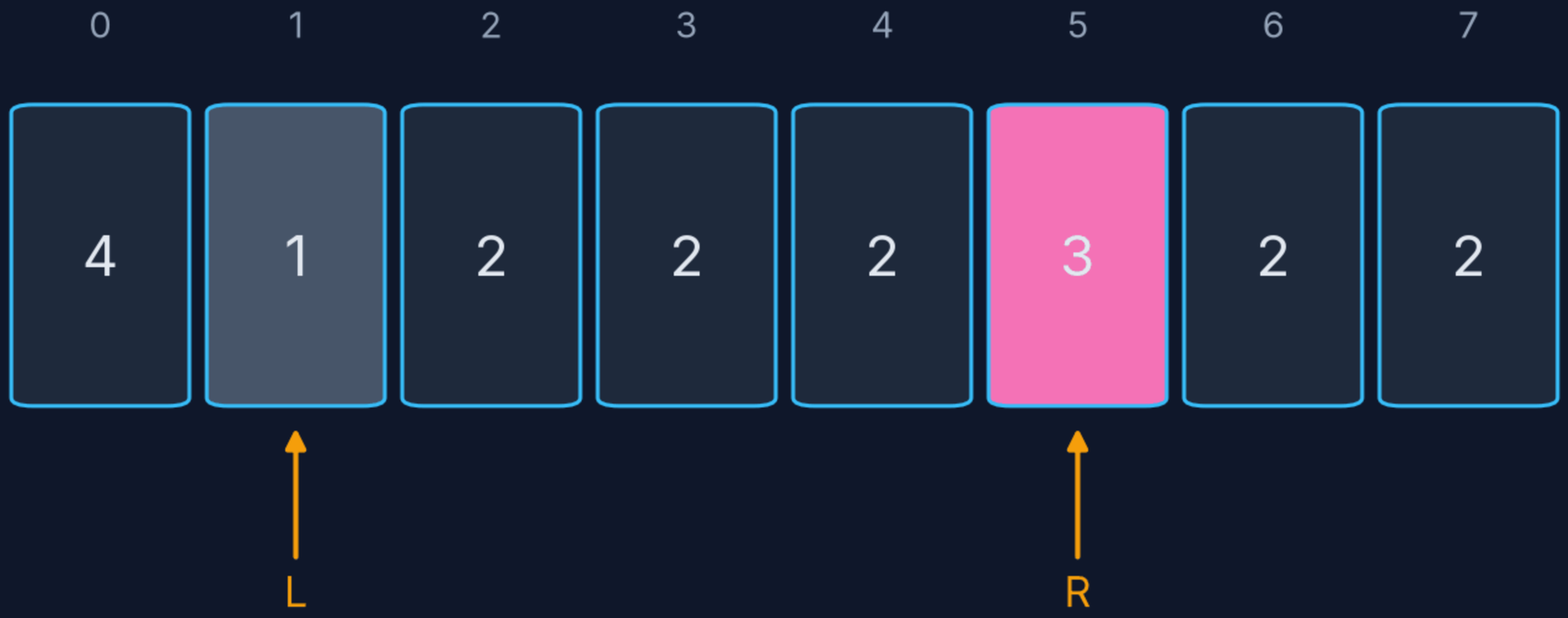
- 1. Left and right pointers
- 2. Skip targets already at the right
- 3. Swap a target on the left with a non-target on the right
- 4. Meet in the middle

Worked example



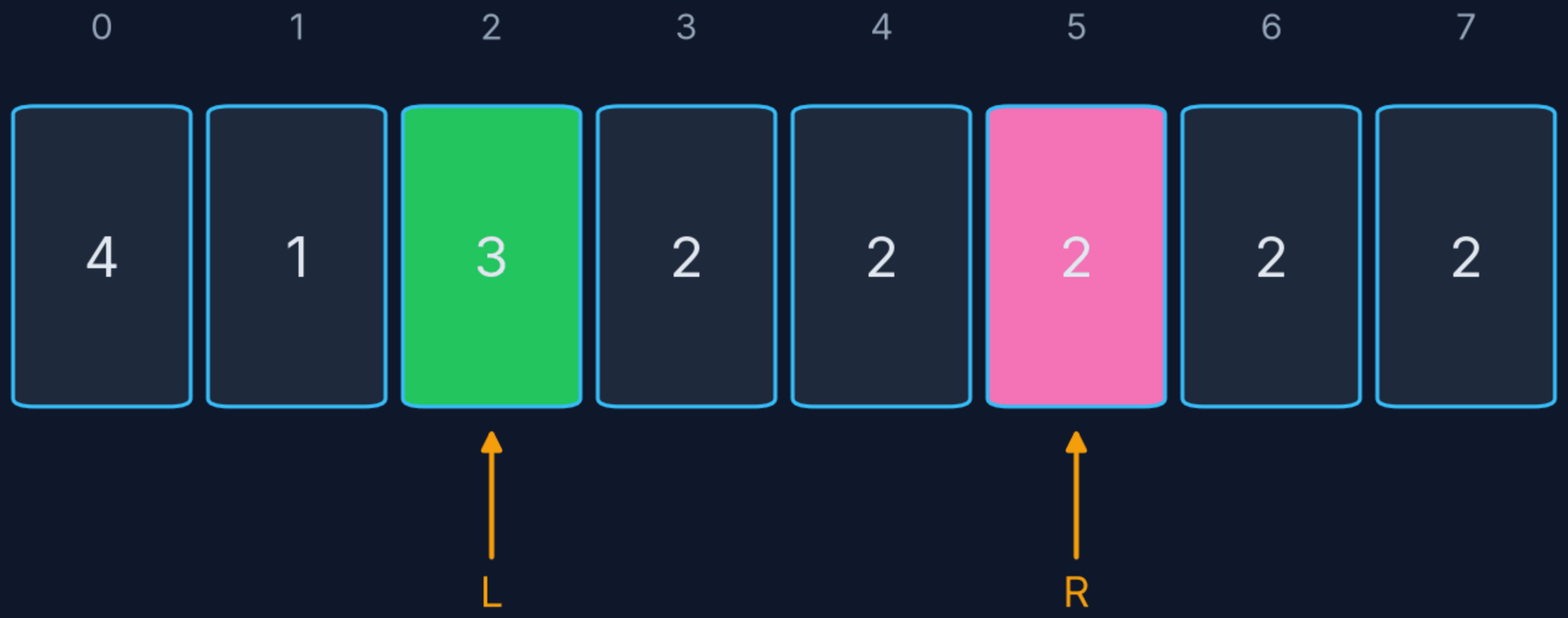
swap a[0] ↔ a[6]

Worked example



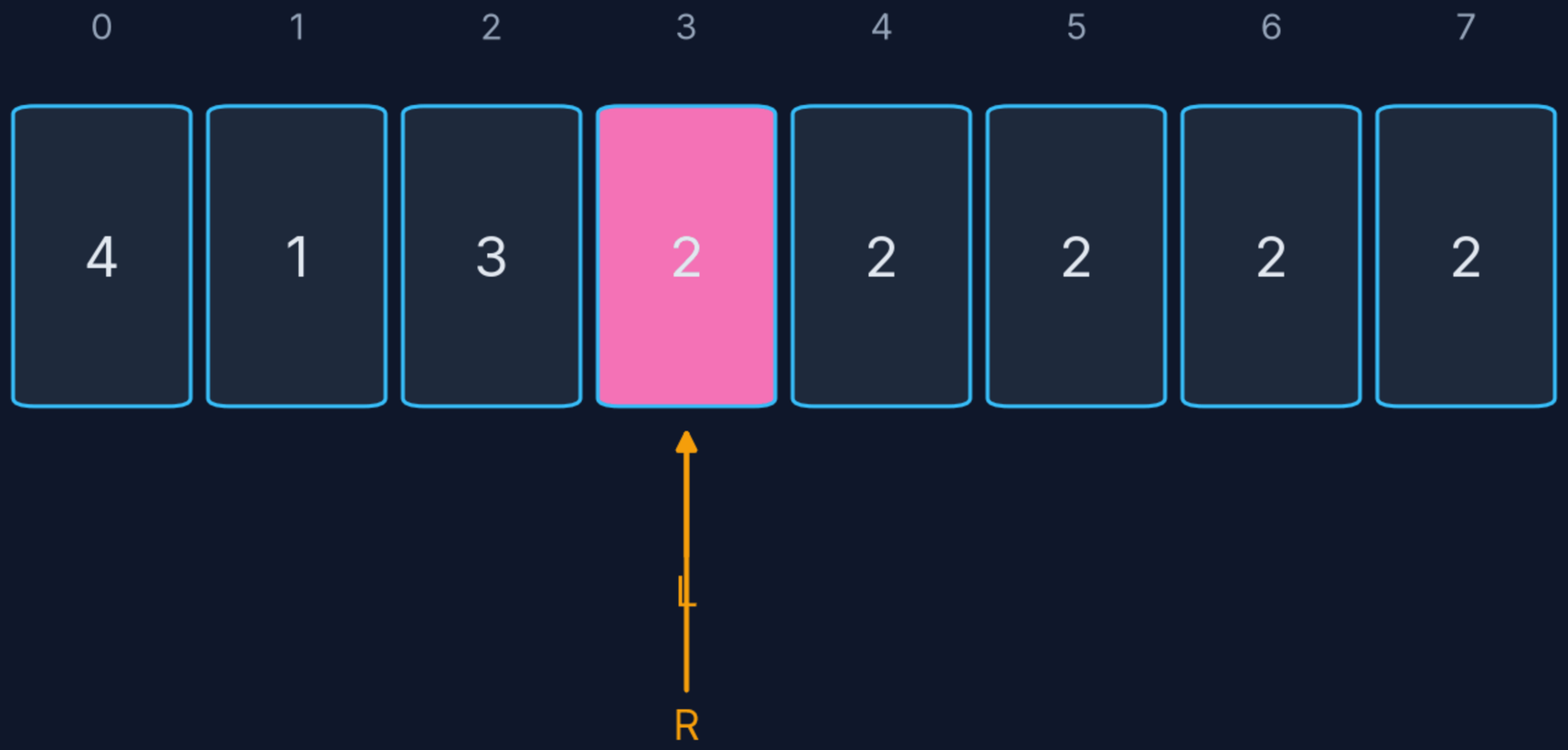
a[1] = 1 ≠ 2, keep

Worked example



swap a[2] ↔ a[5]

Worked example



swap a[3] ↔ a[3]

Java solution

```java
public class MoveElementToEnd {
    public static List<Integer> moveElementToEnd(List<Integer> array, int toMove) {
        int l = 0, r = array.size() - 1;
        while (l < r) {
            while (l < r && array.get(r) == toMove) r--;
            if (array.get(l) == toMove) Collections.swap(array, l, r);
            l++;
        }
        return array;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Order of other elements need not be kept
- All targets or no targets
- Check l < r inside the inner loop

Interview tips & follow-ups

- Follow-up: keep relative order (stable) → write pointer
- Related: Dutch flag partitioning

Recap

- Two pointers: move the target from the left end out of the way by swapping with non-targets from the right.
- Complexity: O(n)